模块一

1.11010011 2.182 3.求n&1,结果为1则是奇数,反之则为偶数

模块二

1.8 29 21 60 2. int get_bit(int x, int n) { return (x >> (n - 1)) & 1; }

3.

int set_bit(int x, int n, int t) { if (t == 1) { return x | (1 << (n - 1)); } else { return x & ~(1 << (n - 1)); } }

4.

int lowest_set_bit(int x) { //补码性质 return x & -x; }

5.

unsigned long long range_and(unsigned long long a, unsigned long long b) { //寻找最大公共前缀 int shift = 0; while (a < b) { a >>= 1; b >>= 1; shift++; } return a << shift; }

位运算和逻辑运算在底层都基于布尔代数。位运算可以看作是将布尔代数的四种基本运算（与、或、非、异或）逐位并行地应用到整个机器字上，逻辑运算则是将整个值先视作为一个布尔量（非零为真，零为假），再做布尔运算。

逻辑右移高位补0，算术右移高位补符号位。现代计算机中有符号整数采用补码存储，在补码中，最高位是符号位：0 表示非负，1 表示负数。因此逻辑右移会改变有符号整数的正负性。

模块三

#include <stdio.h>

int hasCommonChar(const char *s1, const char *s2) { int mask1 = 0; int mask2 = 0;

```
for (int i = 0; s1[i] != '\0'; i++) {  //记录字母是否出现
    mask1 |= (1 << (s1[i] - 'a'));
}

for (int i = 0; s2[i] != '\0'; i++) {
    mask2 |= (1 << (s2[i] - 'a'));
}

return (mask1 & mask2) != 0;         判断是否出现相同字母
```

}

模块四

#include <stdio.h>

int count; // 全局变量，记录方案总数

// n: 棋盘大小 // row: 当前处理的行号（从 0 开始） // col: 列占用位图，第 i 位为 1 表示第 i 列已被占用 // ld: 左对角线占用位图（从左上到右下） // rd: 右对角线占用位图（从右上到左下）

void dfs(int n, int row, int col, int ld, int rd) { if (row == n) { count++; // 所有行都放置完毕，找到一个合法方案 return; }

```
int available = ((1 << n) - 1) & ~(col | ld | rd);     // 计算当前行所有可放置的位置：1 表示可放，0 表示不可放

while (available) {
    int pos = available & -available;         // 取出最低位的1，即当前行最右边可放置的位置

    available -= pos;                //清除该位

    // 递归到下一行
    // 下一行的列占用：col | pos
    // 下一行的左对角线：(ld | pos) << 1 （因为左对角线向右下延伸）
    // 下一行的右对角线：(rd | pos) >> 1 （因为右对角线向左下延伸）
    dfs(n, row + 1, col | pos, (ld | pos) << 1, (rd | pos) >> 1);
}
```

}

int totalNQueens(int n) { count = 0; dfs(n, 0, 0, 0, 0); return count; }

int main() { int n; printf("请输入 N (1-15): "); if (scanf("%d", &n) != 1) { printf("输入错误！\n"); return 1; } if (n < 1 || n > 15) { printf("N 的范围是 1 到 15\n"); return 1; }

```
int result = totalNQueens(n);
printf("N=%d 的方案总数为: %d\n", n, result);
return 0;
```

}

模块五

#include <stdio.h> #include <stdbool.h>

// 定义简单队列结构用于 BFS typedef struct { int pos; // 当前节点编号 (0~15) int dist; // 到达当前节点的最短步数 } Node;

int minStepsToCheese(int walls) { int start = 0; int target = 15;

```
// 如果起点或终点本身是墙，直接不可达
if ((walls & (1 << start)) || (walls & (1 << target))) {
    return -1;
}

// BFS 队列与访问位图
Node queue[16];
int front = 0, rear = 0;
int visited = 0;

// 起点入队并标记已访问
queue[rear++] = (Node){start, 0};
visited |= (1 << start);

// 上、下、左、右四个方向的节点偏移量
int dr[4] = {-1, 1, 0, 0};
int dc[4] = {0, 0, -1, 1};

// TO DO
while (front < rear) {
    Node cur = queue[front++];

    // 到达终点，返回步数
    if (cur.pos == target) {
        return cur.dist;
    }

    int r = cur.pos / 4;   // 当前行
    int c = cur.pos % 4;   // 当前列

    // 遍历四个方向
    for (int i = 0; i < 4; i++) {
        int nr = r + dr[i];
        int nc = c + dc[i];

        // 边界检查
        if (nr < 0 || nr >= 4 || nc < 0 || nc >= 4) {
            continue;
        }

        int new_pos = nr * 4 + nc;

        // 墙壁检查
        if (walls & (1 << new_pos)) {
            continue;
        }

        // 已访问检查
        if (visited & (1 << new_pos)) {
            continue;
        }
```

```
            // 标记已访问并入队
            visited |= (1 << new_pos);
            queue[rear++] = (Node){new_pos, cur.dist + 1};
        }
    }
    // END OF TO DO

    return -1; // 无法到达
```

}

int main() { int walls = (1 << 5) | (1 << 10); int steps = minStepsToCheese(walls); printf("Minimum steps: %d\n", steps); return 0; }